Open in Colab

**After clicking the "Open in Colab" link, copy the notebook to your own Google Drive before getting started, or it will not save your work**

# BYU CS 180 Lab 10: Train and Test ML Models

```
# Dependencies for the lab
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
```

## Introduction:

You will use the `sklearn` API to train various classifiers. Feel free to use any `sklearn`, `numpy`, or `pandas` function to help you with the lab.

The 20 newsgroups dataset comprises newsgroups posts on 20 topics. The 20 categories are:

```
['alt.atheism',
 'comp.graphics',
 'comp.os.ms-windows.misc',
 'comp.sys.ibm.pc.hardware',
 'comp.sys.mac.hardware',
 'comp.windows.x',
 'misc.forsale',
 'rec.autos',
 'rec.motorcycles',
 'rec.sport.baseball',
 'rec.sport.hockey',
 'sci.crypt',
 'sci.electronics',
 'sci.med',
 'sci.space',
 'soc.religion.christian',
 'talk.politics.guns',
 'talk.politics.mideast',
 'talk.politics.misc',
 'talk.religion.misc']
```

The posts are raw text. For example:

**Post:**

```
From: lerxst@wam.umd.edu (where's my thing)
Subject: WHAT car is this!?
Nntp-Posting-Host: rac3.wam.umd.edu
Organization: University of Maryland, College Park
Lines: 15

 I was wondering if anyone out there could enlighten me on this car I saw
the other day. It was a 2-door sports car, looked to be from the late 60s/
early 70s. It was called a Bricklin. The doors were really small. In addition,
the front bumper was separate from the rest of the body. This is
all I know. If anyone can tellme a model name, engine specs, years
of production, where this car is made, history, or whatever info you
have on this funky looking car, please e-mail.

Thanks,
- IL
   ---- brought to you by your neighborhood Lerxst ----
```

**Target:**

```
'rec.autos'
```

Your task will be to build a classifier to predict the category of posts. You will do the following in this notebook:

## Exercise 1: Retrieve the Data

Download `fetch_20newsgroups` dataset

```
from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups()
docs = data['data']
targets = data["target"]
names = data.target_names
```

Print off the first document and its target classification to make sure its been downloaded correctly. What is the target name of the first document?

```
#Enter code here.
print(docs[0])
names[targets[0]]
```

```
From: lerxst@wam.umd.edu (where's my thing)
Subject: WHAT car is this!?
Nntp-Posting-Host: rac3.wam.umd.edu
Organization: University of Maryland, College Park
Lines: 15

 I was wondering if anyone out there could enlighten me on this car I saw
the other day. It was a 2-door sports car, looked to be from the late 60s/
early 70s. It was called a Bricklin. The doors were really small. In addition,
the front bumper was separate from the rest of the body. This is
all I know. If anyone can tellme a model name, engine specs, years
of production, where this car is made, history, or whatever info you
have on this funky looking car, please e-mail.

Thanks,
- IL
   ---- brought to you by your neighborhood Lerxst ----
```

```
'rec.autos'
```

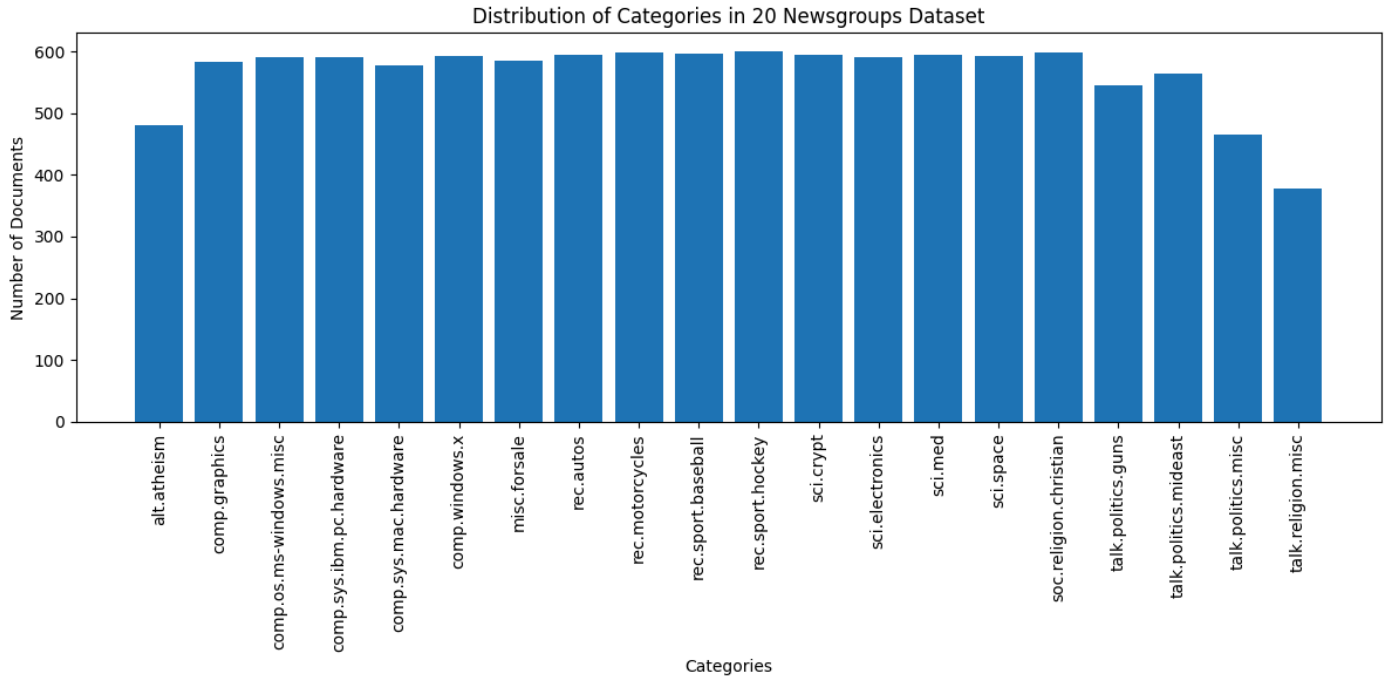
## Exercise 2: Understand the Data

Perform some exploratory analysis on your data. How many documents are there? What does the class balance look like? Specifically, plot a bar chart showing distribution of the categories (classes). Be sure to label the axis using the target names.

```
#Enter code here
# Count
num_docs = len(docs)
print(f"Number of documents: {num_docs}")

# class distribution
unique_targets, counts = np.unique(targets, return_counts=True)
class_distribution = dict(zip(unique_targets, counts))

print("Class Distribution:")
for target, count in class_distribution.items():
    print(f"Class {names[target]}: {count} documents")

# Plot
plt.figure(figsize=(12, 6))
plt.bar(names, counts)
plt.xlabel("Categories")
plt.ylabel("Number of Documents")
plt.title("Distribution of Categories in 20 Newsgroups Dataset")
```

```
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

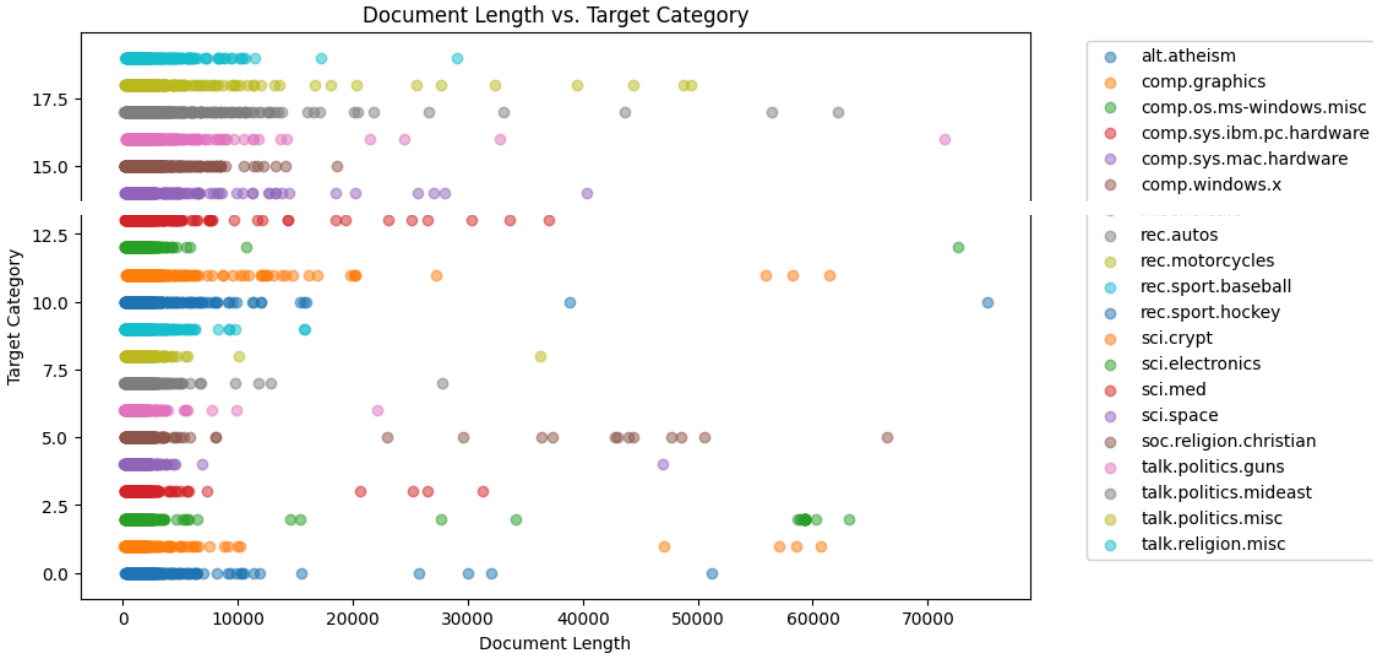
```
Number of documents: 11314
Class Distribution:
Class alt.atheism: 480 documents
Class comp.graphics: 584 documents
Class comp.os.ms-windows.misc: 591 documents
Class comp.sys.ibm.pc.hardware: 590 documents
Class comp.sys.mac.hardware: 578 documents
Class comp.windows.x: 593 documents
Class misc.forsale: 585 documents
Class rec.autos: 594 documents
Class rec.motorcycles: 598 documents
Class rec.sport.baseball: 597 documents
Class rec.sport.hockey: 600 documents
Class sci.crypt: 595 documents
Class sci.electronics: 591 documents
Class sci.med: 594 documents
Class sci.space: 593 documents
Class soc.religion.christian: 599 documents
Class talk.politics.guns: 546 documents
Class talk.politics.mideast: 564 documents
Class talk.politics.misc: 465 documents
Class talk.religion.misc: 377 documents
```




```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
from sklearn.datasets import fetch_20newsgroups

# Assuming 'data' is already loaded as in the provided code
# data = fetch_20newsgroups()
# docs = data['data']
# targets = data["target"]
# names = data.target_names

# Calculate the average length of documents
doc_lengths = [len(doc) for doc in docs]
avg_doc_length = np.mean(doc_lengths)
print(f"Average document length: {avg_doc_length:.2f} characters")

# Analyze word frequency (top 10 most frequent words)
from collections import Counter
```

```
# Combine all documents into a single string
all_text = " ".join(docs)
# Remove punctuation and convert to lowercase for accurate counting
all_text = ''.join(c for c in all_text if c.isalnum() or c.isspace()).lower()
words = all_text.split()
word_counts = Counter(words)
top_10_words = word_counts.most_common(10)

print("Top 10 most frequent words:")
for word, count in top_10_words:
    print(f"{word}: {count}")

# Create a word cloud (optional, requires wordcloud library)
# !pip install wordcloud
# from wordcloud import WordCloud

# wordcloud = WordCloud(width=800, height=400, background_color='white').generate_from_frequencies(word_counts)
# plt.figure(figsize=(10, 5))
# plt.imshow(wordcloud, interpolation='bilinear')
# plt.axis('off')
# plt.show()


# Example:  Explore the relationship between document length and target category
plt.figure(figsize=(10, 6))
for i in range(len(names)):
  plt.scatter(np.array(doc_lengths)[targets==i], np.zeros_like(np.array(doc_lengths)[targets==i]) + i, label=names[i], alpha=0.5

plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left')
plt.ylabel("Target Category")
plt.xlabel("Document Length")
plt.title("Document Length vs. Target Category")

plt.show()
```

```
Average document length: 1949.31 characters
Top 10 most frequent words:
the: 146332
to: 72647
of: 68883
a: 60943
and: 57451
in: 48661
is: 43278
i: 41559
that: 37736
it: 29529
```



## Enter comments about your EDA below.

It looks like there are less posts under tha labels atheism, policts/guns, politics/misc and religion/music.

The word "the" is the most frequent;y used word, this is not suprising.

Most of the the documents are less than 10,000 for the majority of the different categories.

## Exercise 3: Engineer the Data

Extract features from raw text. You are free to be creative here. There is no correct answer. Try and think of the features (input) that are most predictive of the category. Some helpful sklearn feature engineering code can be found here (I'd recommend reading through the documentation before diving in and trying to get these tools to work):

- CountVectorizer
- TFIDF Text Features

```
#Enter code here to engineer the data
```

```
# prompt: Extract features from raw text

from sklearn.feature_extraction.text import TfidfVectorizer

# Create a TfidfVectorizer object
vectorizer = TfidfVectorizer(max_features=5000, stop_words='english') # Limit to top 5000 features

# Fit and transform the document data
X = vectorizer.fit_transform(docs)

# Get feature names
feature_names = vectorizer.get_feature_names_out()

# Print some info about the features
print(f"Number of features: {len(feature_names)}")
print("Example features:", feature_names[:10]) # Print the first 10

# Now X is a matrix where each row represents a document and each column represents a feature (TF-IDF score)
# You can use this X as input to your classification models
```

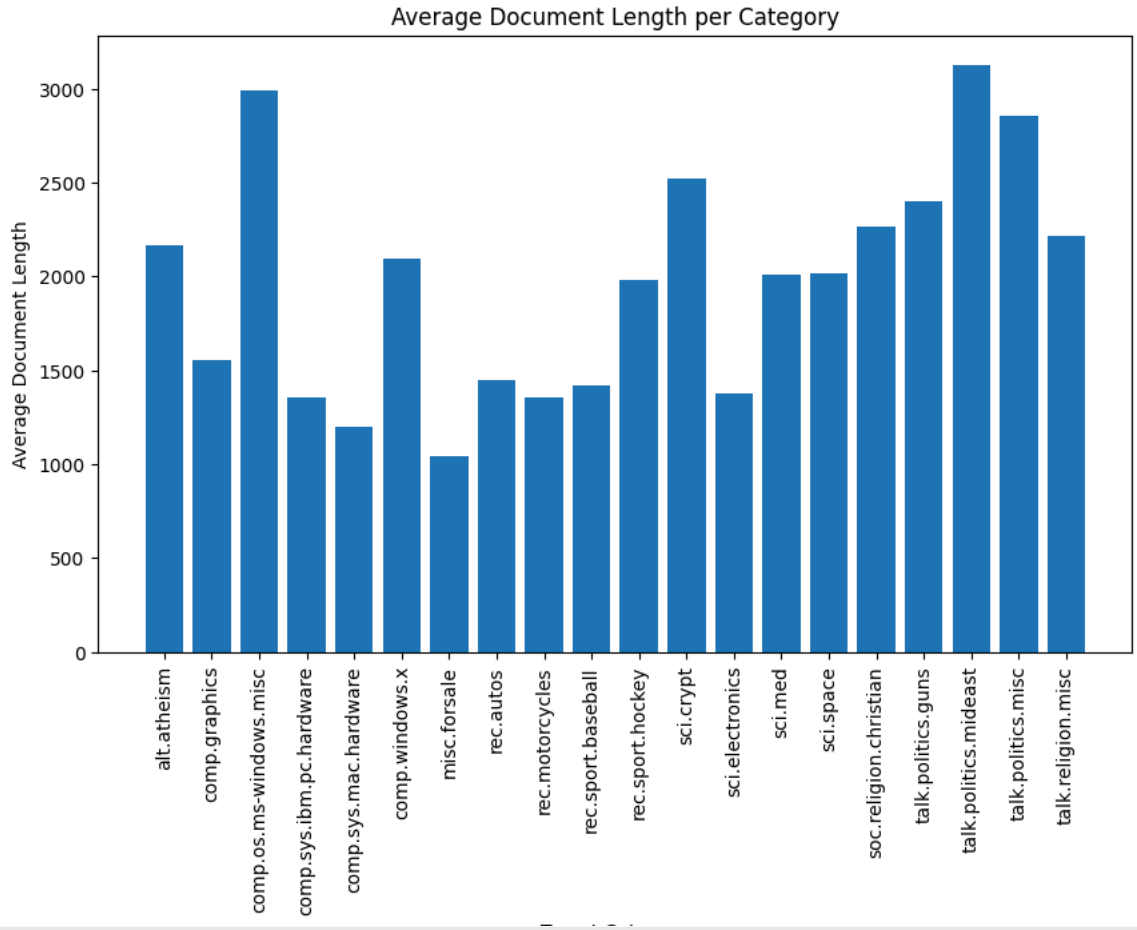
```
Number of features: 5000
Example features: ['00' '000' '01' '02' '03' '04' '040' '05' '06' '07']
```

```
# prompt: extract more data from the raw text

# Analyze word frequency (top 20 most frequent words) across each category

for i in range(len(names)):
    # Combine documents of the current category into a single string
    category_docs = [docs[j] for j in range(len(docs)) if targets[j] == i]
    category_text = " ".join(category_docs)

    # Remove punctuation and convert to lowercase
    category_text = ''.join(c for c in category_text if c.isalnum() or c.isspace()).lower()

    words = category_text.split()
    word_counts = Counter(words)

    top_20_words = word_counts.most_common(20)
    print(f"\nTop 20 words for category: {names[i]}")
    for word, count in top_20_words:
      print(f"{word}: {count}")
```

```
Top 20 words for category: alt.atheism
the: 7231
of: 4250
to: 4189
is: 3530
a: 3220
```

```
that: 3049
and: 2758
in: 2699
i: 2149
it: 2047
you: 1824
not: 1580
be: 1298
are: 1298
this: 1215
for: 1199
have: 1161
from: 1052
as: 1017
but: 918

Top 20 words for category: comp.graphics
the: 4535
to: 2639
a: 2628
of: 2467
and: 2339
is: 1742
i: 1670
in: 1645
for: 1594
it: 1252
from: 1177
you: 970
that: 968
on: 888
this: 841
be: 721
or: 709
with: 705
have: 662
lines: 640

Top 20 words for category: comp.os.ms-windows.misc
the: 3803
maxaxaxaxaxaxaxaxaxaxaxaxaxaxaxaxax: 3307
i: 2251
to: 2161
a: 2017
of: 1568
and: 1549
is: 1464
it: 1258
in: 1241
for: 1132
windows: 988
```

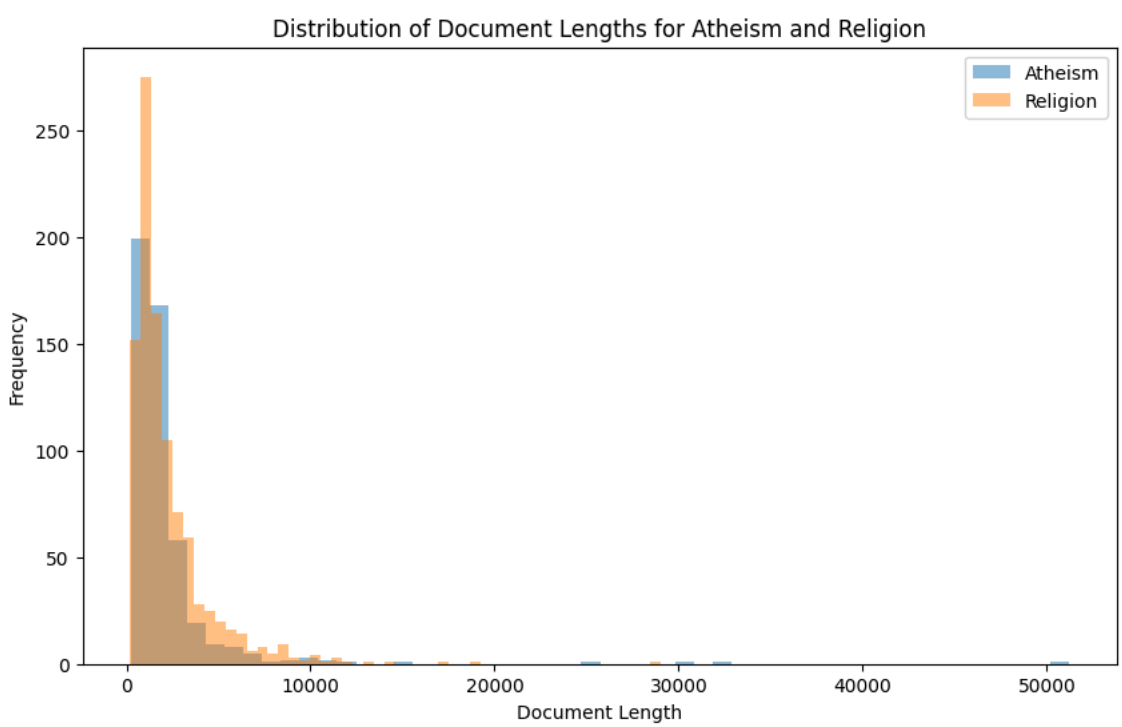
```
avg_doc_length_per_category = []
for i in range(len(names)):
  category_docs = [docs[j] for j in range(len(docs)) if targets[j] == i]
  doc_lengths = [len(doc) for doc in category_docs]
  avg_doc_length = np.mean(doc_lengths)
  avg_doc_length_per_category.append(avg_doc_length)

plt.figure(figsize=(10,6))
plt.bar(names, avg_doc_length_per_category)
plt.xticks(rotation=90)
plt.ylabel("Average Document Length")
plt.xlabel("Target Category")
plt.title("Average Document Length per Category")
plt.show()
```




It is interesting but not suprising that the 2 of the 3 document types that have the longest average length are those that involve politics. It is also interesting that Atheism and religion are similar.

```
import matplotlib.pyplot as plt

# Assuming 'names' and 'targets' are defined from the previous code
# and 'data' is the fetch_20newsgroups dataset

atheism_index = names.index('alt.atheism')
religion_indices = [names.index(name) for name in names if 'religion' in name]

atheism_lengths = [len(doc) for i, doc in enumerate(data['data']) if targets[i] == atheism_index]
religion_lengths = [len(doc) for i, doc in enumerate(data['data']) if targets[i] in religion_indices]

plt.figure(figsize=(10, 6))
plt.hist(atheism_lengths, alpha=0.5, label='Atheism', bins=50)
plt.hist(religion_lengths, alpha=0.5, label='Religion', bins=50)

plt.xlabel('Document Length')
plt.ylabel('Frequency')
plt.title('Distribution of Document Lengths for Atheism and Religion')
plt.legend()
plt.show()

# Calculate basic statistics
print("Atheism:")
print("Mean:", np.mean(atheism_lengths))
print("Std:", np.std(atheism_lengths))

print("\nReligion:")
print("Mean:", np.mean(religion_lengths))
print("Std:", np.std(religion_lengths))

# Convert to numpy arrays for numerical operations
atheism_np = np.array(atheism_lengths)
religion_np = np.array(religion_lengths)
```

```
# Find the minimum length
min_len = min(len(atheism_np), len(religion_np))

# Truncate arrays to the minimum length for correlation analysis
atheism_np = atheism_np[:min_len]
religion_np = religion_np[:min_len]

# Calculate correlation coefficient
correlation = np.corrcoef(atheism_np, religion_np)[0, 1]
print(f"\nCorrelation coefficient between atheism and religion document lengths: {correlation}")
```




```
Atheism:
Mean: 2167.3020833333335
Std: 3577.9652350643364

Religion:
Mean: 2243.7663934426228
Std: 2276.242644035635
```

## Exercise 4: Split the Data

Split your data into train and test splits. Use an 80%/20% split rule for train and test respectively.

```
#Enter code here to split the dataset
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, targets, test_size=0.2, random_state=42)
```

## Exercise 5: Train the Model

Using sklearn, train the following classifiers using the training set:

- KNN
- Decision Tree
- Logistic Regression
- Random Forests
- At least one other classification method NOT discussed in class.

Use cross validation to perform hyperparameter tuning.

```
# Train the model using the KNN Cluster Algorithm


from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

# Create a KNN classifier
knn = KNeighborsClassifier()

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'n_neighbors': [3, 5, 7, 9],  # Example values for k
    'weights': ['uniform', 'distance'],  # Example weighting methods
    'metric': ['euclidean', 'manhattan']  # Example distance metrics
}

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy') # Use 5-fold cross-validation
grid_search.fit(X_train, y_train)

# Get the best model
best_knn = grid_search.best_estimator_

# Print the best hyperparameters and score
print("Best hyperparameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)

# Evaluate on the test set
test_accuracy = best_knn.score(X_test, y_test)
print(f"Test accuracy of best KNN model: {test_accuracy}")
```

```
Best hyperparameters: {'metric': 'euclidean', 'n_neighbors': 3, 'weights': 'distance'}
Best cross-validation score: 0.7473218605757938
Test accuracy of best KNN model: 0.7834732655766682
```

```
# Train the model using the Decision Tree Algorithm


# prompt: Train the model using the Decision Tree Algorithm

from sklearn.tree import DecisionTreeClassifier

# Create a Decision Tree classifier
dt = DecisionTreeClassifier()

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'criterion': ['gini', 'entropy'],  # Splitting criteria
    'max_depth': [None, 10, 20, 30],  # Maximum depth of the tree
    'min_samples_split': [2, 5, 10],  # Minimum samples required to split an internal node
    'min_samples_leaf': [1, 2, 4]  # Minimum samples required to be at a leaf node
}

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best model
best_dt = grid_search.best_estimator_

# Print the best hyperparameters and score
print("Best hyperparameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)

# Evaluate on the test set
test_accuracy = best_dt.score(X_test, y_test)
print(f"Test accuracy of best Decision Tree model: {test_accuracy}")
```

```
Best hyperparameters: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best cross-validation score: 0.6315320432836777
Test accuracy of best Decision Tree model: 0.6513477684489616
```

```
# Train the model using the Logistic Regression Algorithm


# prompt: Train the model using the Logistic Regression Algorithm

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Create a Logistic Regression classifier
lr = LogisticRegression(max_iter=10) # Increased max_iter

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10],  # Inverse of regularization strength
    'penalty': ['l1', 'l2'],  # Regularization type
    'solver': ['liblinear', 'saga'] # Solvers compatible with both penalties
}

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(lr, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best model
best_lr = grid_search.best_estimator_

# Print the best hyperparameters and score
print("Best hyperparameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)

# Evaluate on the test set
test_accuracy = best_lr.score(X_test, y_test)
print(f"Test accuracy of best Logistic Regression model: {test_accuracy}")
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was reached wl
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was reached wl
  warnings.warn(
Best hyperparameters: {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}
Best cross-validation score: 0.8736066579009186
Test accuracy of best Logistic Regression model: 0.8828988068935042
/usr/local/lib/python3.11/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear failed to converge, inc
  warnings.warn(
```

```
# Train the model using the Random Forest Algorithm
from sklearn.ensemble import RandomForestClassifier

# Create a Random Forest classifier
rf = RandomForestClassifier()

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'n_estimators': [100, 200, 300],  # Number of trees in the forest
    'max_depth': [None, 10,
                  20, 30],  # Maximum depth of each tree
    'min_samples_split': [2, 5, 10],  # Minimum samples required to split an internal node
    'min_samples_leaf': [1,4, 8]  # Minimum samples required to be at a leaf node
}

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best model
best_rf = grid_search.best_estimator_

# Print the best hyperparameters and score
print("Best hyperparameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)

# Evaluate on the test set
test_accuracy = best_rf.score(X_test, y_test)
print(f"Test accuracy of best Random Forest model: {test_accuracy}")
```

```
Best hyperparameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}
Best cross-validation score: 0.8150488573511782
Test accuracy of best Random Forest model: 0.8347326557666814
```

```
# Train the model using some other method
```

```
from sklearn.model_selection import GridSearchCV
import xgboost as xgb

xgb_model = xgb.XGBClassifier(objective='multi:softmax', num_class=20, random_state=42)

param_grid = {
    'learning_rate': [0.1],
    'max_depth': [3, 5],
    'n_estimators': [100],
}

grid_search = GridSearchCV(estimator=xgb_model, param_grid=param_grid, cv=2, scoring='accuracy', verbose=1, n_jobs=-1)
grid_search.fit(X_train, y_train)

best_xgb_model = grid_search.best_estimator_
print("Best hyperparameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)
print("Test accuracy:", best_xgb_model.score(X_test, y_test))
```

```
Fitting 2 folds for each of 2 candidates, totalling 4 fits
Best hyperparameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100}
Best cross-validation score: 0.7679810694745888
Test accuracy: 0.8064516129032258
```

## Exercise 6: Evaluate the Model

Compare the performance of your classifiers on the test set. Discuss which one performs the best.

```
# Evaluate the KNN Cluster model
```

Best cross-validation score: 0.7473218605757938

Test accuracy of best KNN model: 0.7834732655766682

```
# Evaluate the Decision Tree model
```

Best cross-validation score: 0.6315320432836777

Test accuracy of best Decision Tree model: 0.6513477684489616

```
# Evaluate the Logistic Regression model
```

Best cross-validation score: 0.8736066579009186

Test accuracy of best Logistic Regression model: 0.8828988068935042

```
# Evaluate the Random Forest Model
```

Best cross-validation score: 0.8150488573511782

Test accuracy of best Random Forest model: 0.8347326557666814

```
# Evaluate the other chosen model
# XGboost
```

Best cross-validation score: 0.7679810694745888

Test accuracy: 0.8064516129032258

Discuss which one performs the best:

is is difficult to determine which model performed best as each had a different metric. Due to the complexity of the models and the differing run times (I had one run for ~2 hr.). It was interesting to me however to see that even with such low metrics the XGBoost model did quite well. With the given metrics, the Logistic regression model had both the highest cross-validation score and the highest accuracy.